

Binary Trees - used for implementing containers.

- not the "usual" operations (adding, deleting, etc) but TRAVERSALS.

Depth first
 preorder ①
 inorder
 postorder

Level order

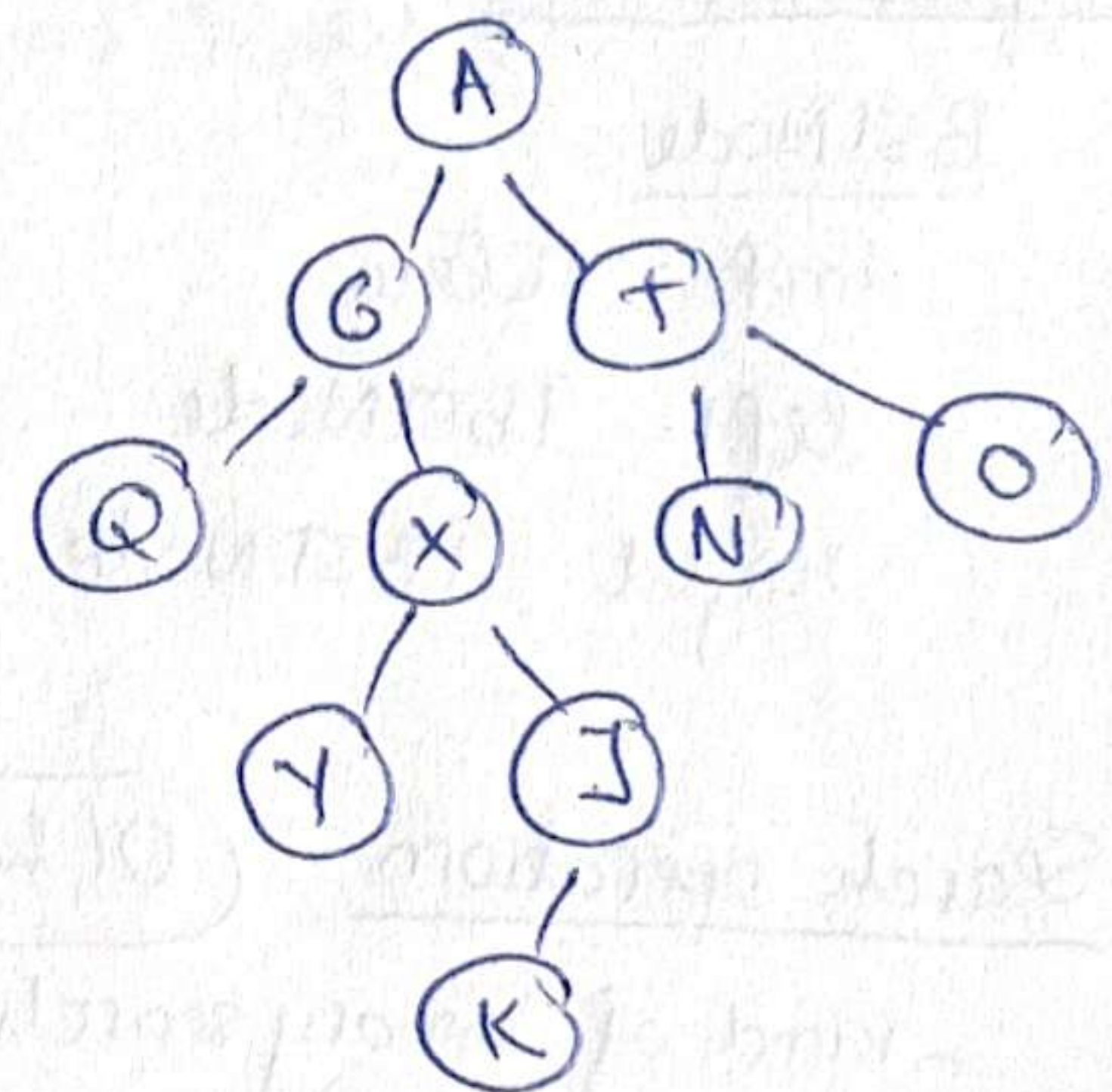
Representation

array
 first pos = root of tree
 disadvantage: space.
 linked representations
 LL on array → extended for binary trees.

① PREORDER (DFS)

- visit root
- traverse left subtree
- traverse right subtree

R - L - R



A G Q X Y J K T N O

② INORDER

- visit left
- root
- right

L - R - R

Q G Y X K J A N T O

③ POSTORDER

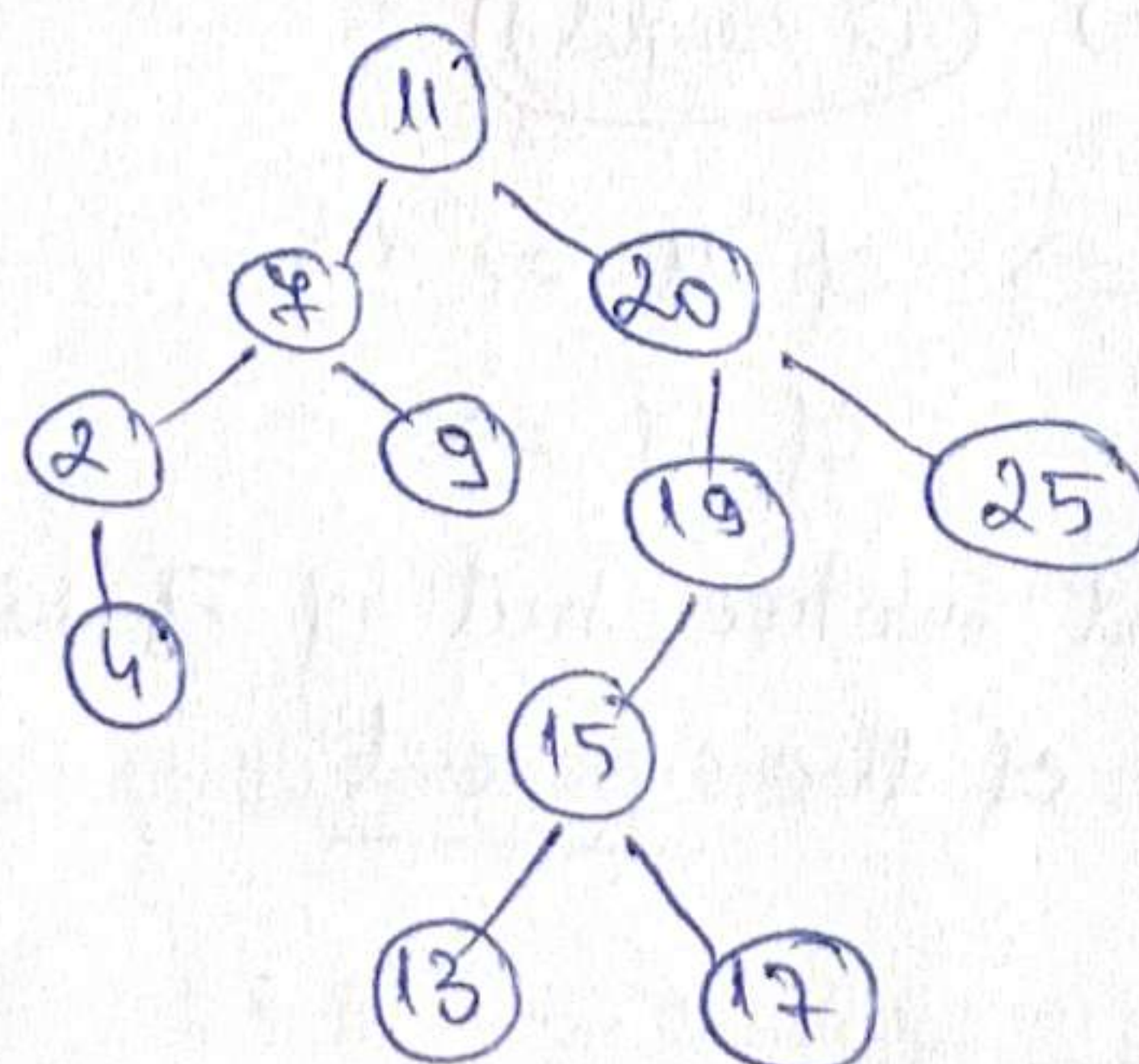
L - R - R

Q Y K J X G N O T A

Binary Search Tree

⊕ extra property: left subtree value $\leq$...
 right subtree value $\geq$...

ex:

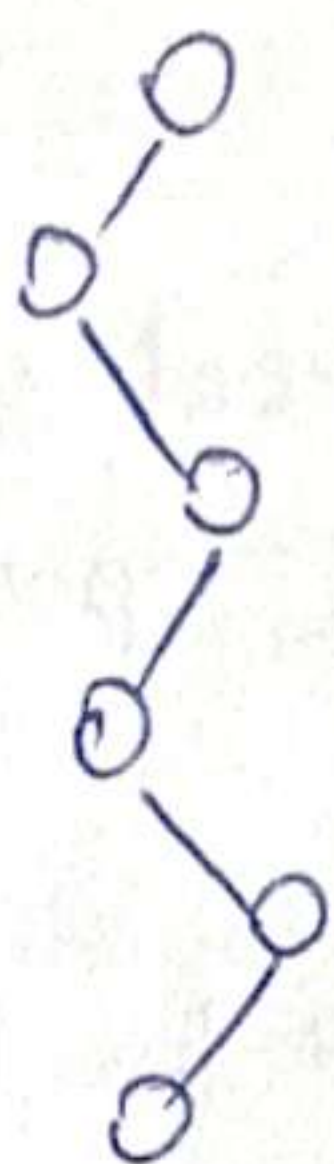


Level order? ("left-root-right") inorder.

2 4 7 9 (11) 13 15 17 19 20 25

Height of a bin. search tree with N nodes $\in (\log_2 N, N)$

Degenerate tree:



Representation

BSTNode:

info: TElem

left: TBSTNode

right: TBSTNode

Binary Search Tree

root: TBSTNode

• Search operation

$O(\text{height})$

- kind of binary search

- WC

- AC

- BC

Implementation

→ recursively / iterative

• Insert operation

$O(\text{height})$

→ DON'T CHANGE THE EXISTING STRUCTURE

→ will be added as a LEAF node.

(14) → only place available is on the right side of (13)?

• Find min node ⇒ start from root ⇒ go left as much as possible.

max ⇒ ... ⇒ right ...

parent of a node ⇒ $O(\text{height})$

successor of node ⇒ of 11 ⇒ 13

of 17 ⇒

→ usually on right subtree, but if $\nexists$ right subtree, we need the perspective of the parent.

- Remove operation (two ways) $\rightarrow$ successor (if no duplicates)
 $\rightarrow$ predecessor

25 $\Rightarrow$ make right side of (20) to point to Nil
 (leaf)

11 $\Rightarrow$ ~ heap $\Rightarrow$ put a $\neq$ value there $\Rightarrow$ (9) or (13)
 (root)

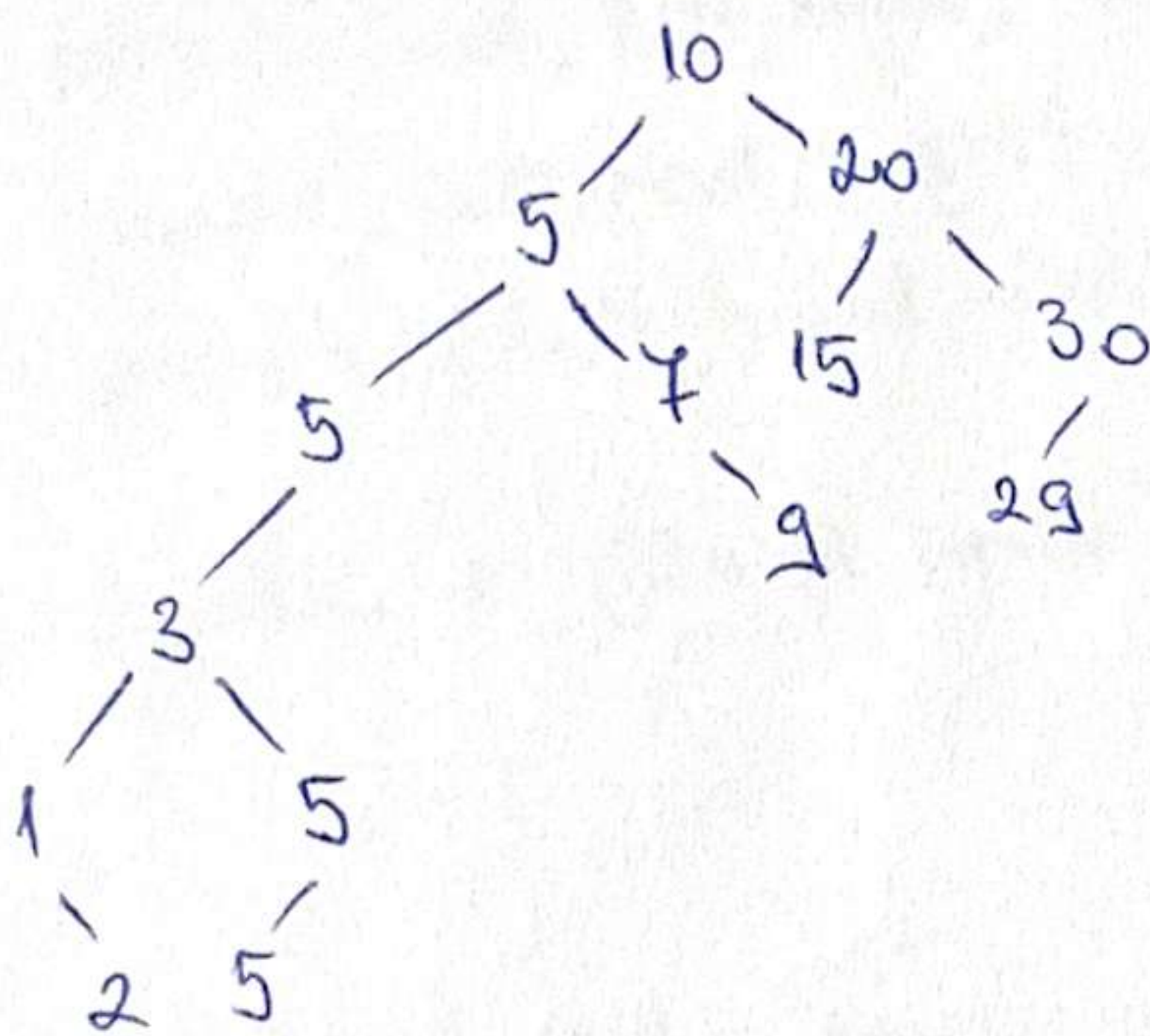
replace with predecessor/successor?

Sorted list on a Binary Search Tree

Binary Search Tree with duplicates

$\leq \Rightarrow$ left. ($=$)

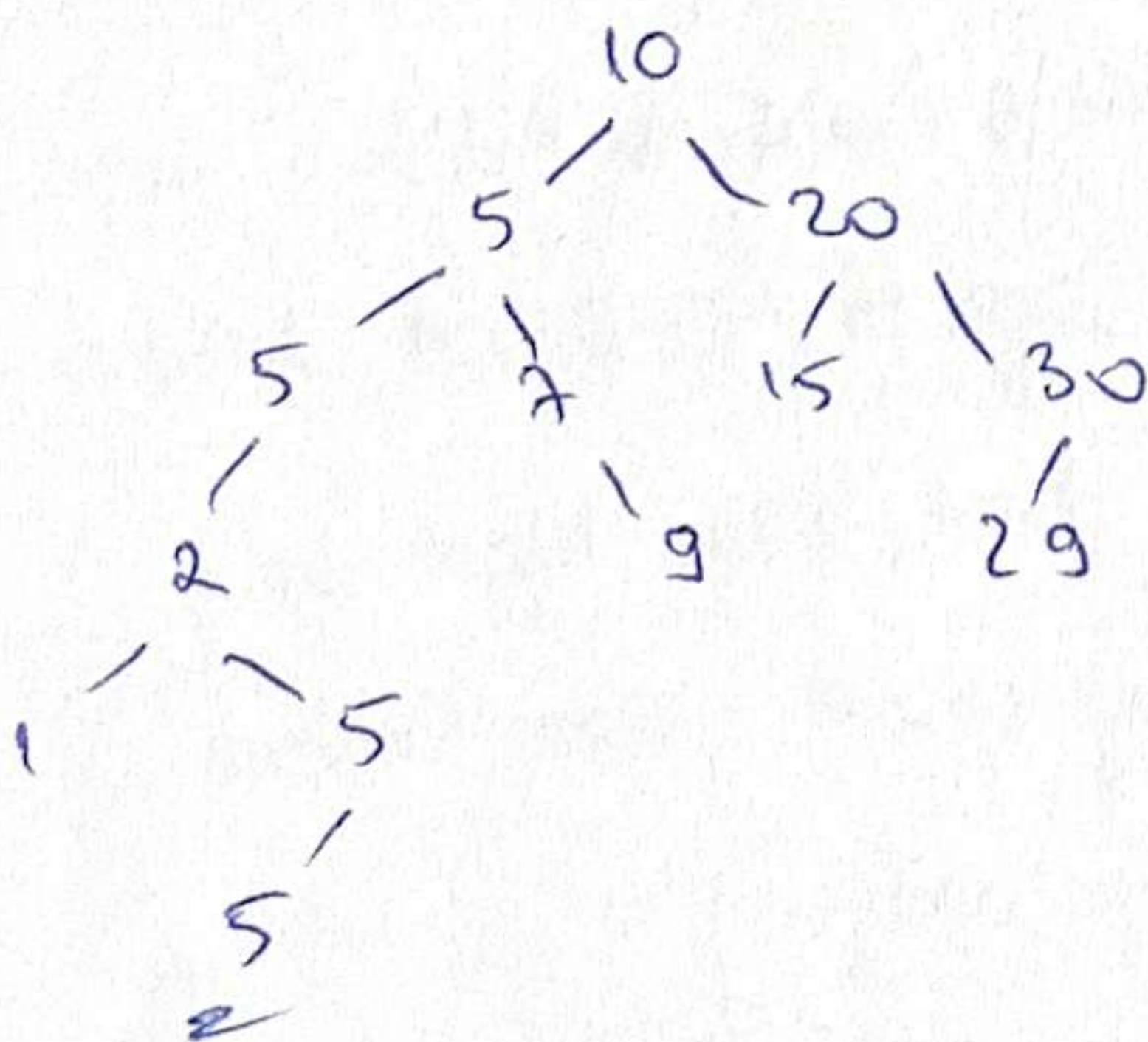
$\Rightarrow$ right!



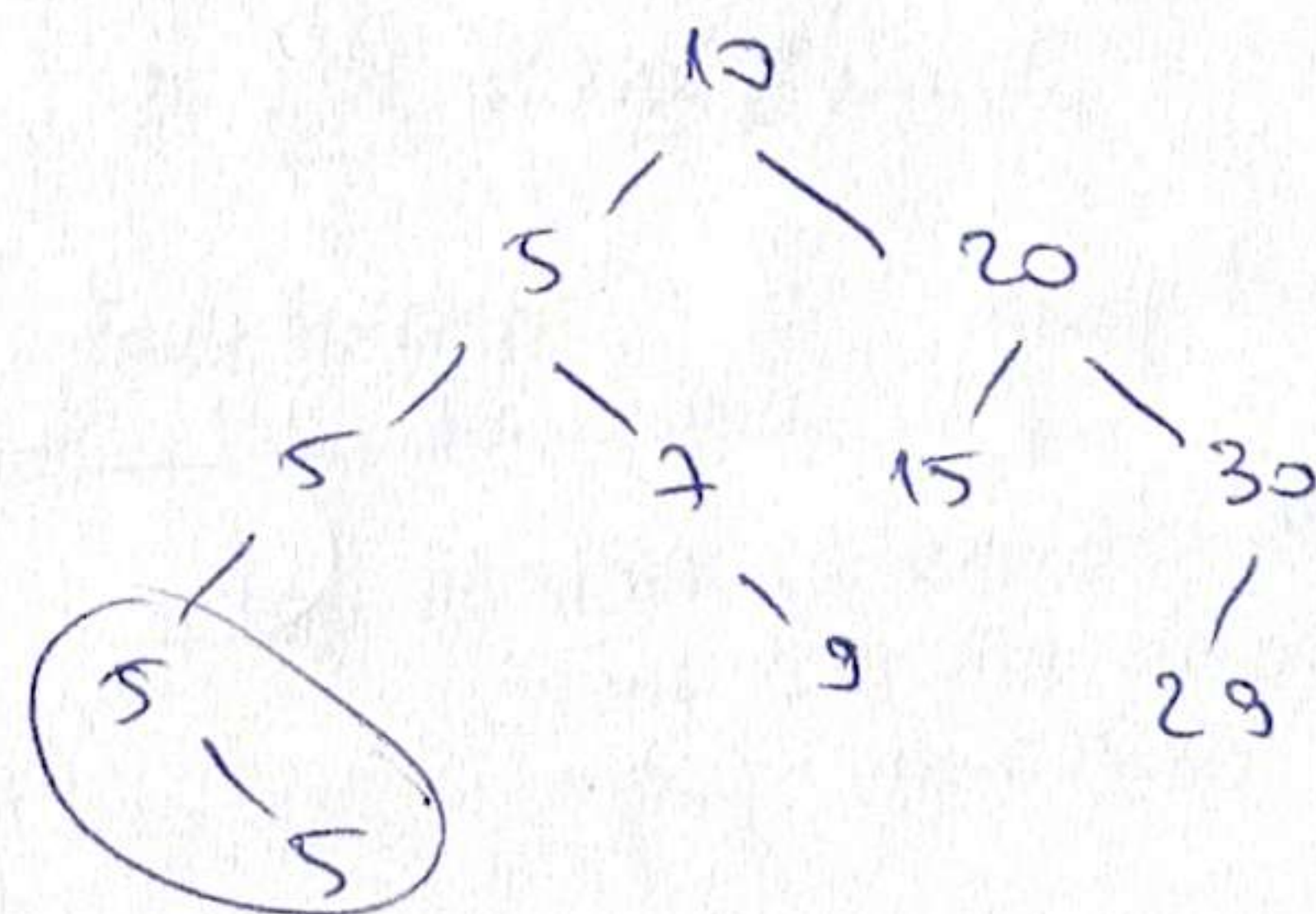
- Count occurrences $\rightarrow$ search, but don't stop when elem found.

show the two possible trees after node (11) removal.

replace w predecessor



replace w successor



- Traverse tree $\rightarrow$ preorder traversal $\Rightarrow$ recursive is easier.

non-recursive: DFS $\rightarrow$ empty stack $\rightarrow$ root node

$\rightarrow$ push right child

$\rightarrow$ push left child.

Inorder implemented recursively

